PROJECT PHASE 4

Ayyub Shaffy (am12321), Matija Susic (ms14012)

1. Architecture and Design

High-Level Structure

In Phase 4, we upgraded the multi-threaded server to simulate a **Single-CPU Scheduler**. While the server still spawns a thread for every connected client to handle network I/O, the actual execution of commands is now serialized to simulate a single processor environment.

We implemented a **Centralized Scheduler Architecture**:

- **Client Threads:** Handle communication and parsing, but they do not execute commands immediately. Instead, they package requests into `Job` structs and add them to a waiting queue.
- **Scheduler Thread:** A dedicated thread that manages the queue, selects the next job based on the SRJF+RR algorithm, and grants the "CPU" (permission to execute) to a specific client thread.

The file structure was updated:

- **`scheduler.c` / `scheduler.h`**: New files containing the `Job` struct definition, the priority queue linked list, and the core scheduling logic (SRJF, preemption checks, and timeline generation).
- **`server.c`**: Refactored to launch the `scheduler_thread_func` on startup. The `client_thread_func` was modified to use Condition Variables to wait for execution turns instead of running immediately.
- **`demo.c`**: A helper program created to simulate long-running processes for testing time-slicing.

Key Design Decisions

1. The Centralized Scheduler Thread We created a dedicated thread (`scheduler_thread_func`) that runs an infinite loop, monitoring the job queue.

- **Reason:** This separates the *decision-making* (who runs next?) from the *execution* (running the process). It simplifies synchronization by having a single authority on the system's state.

2. Condition Variables for Flow Control Each `Job` struct contains its own `pthread_cond_t`. When a client thread submits a job, it sleeps on this condition variable. The scheduler signals this specific variable when it is that job's turn.

- **Reason:** This avoids "busy waiting" (CPU polling). Client threads consume zero CPU resources while waiting in the queue, perfectly simulating a waiting process state.

3. Process Control via Signals (SIGSTOP/SIGCONT) For program commands (like `./demo`), we do not kill and restart the process when time runs out. instead, we use `kill(pid, SIGSTOP)` to pause it and `kill(pid, SIGCONT)` to resume it.

- **Reason:** This is required to maintain the state of the process. Restarting `demo` from the beginning every time would violate the requirement to resume execution from where it left off.

4. Hybrid Scheduling Logic We implemented a prioritized selection logic. "Shell Commands" (e.g., ls, pwd) are assigned a burst time of `-1` and treated as non-preemptive high-priority tasks. "Program Commands" (e.g., ./demo, ./hello) are preemptive and follow Shortest Remaining Job First (SRJF).

- **Reason:** This ensures the UI remains responsive (instant shell output) even when heavy computation (long demo jobs) is happening in the background.

Data Structures

The Job Struct We introduced a `Job` struct to track the execution state of a client request:

```
typedef struct Job {
    int id;                 // Client ID
    int socket_fd;          // Connection to client
    bool is_shell_cmd;      // Priority flag
    int total_time;         // Predicted burst
    int remaining_time;     // For SRJF
    pid_t pid;              // Child process ID (for SIGSTOP/CONT)
    int pipe_fd;            // To read output chunks
    bool started;           // Has fork() happened yet?
    int rounds_run;         // To calculate Quantum (3s vs 7s)

    // Synchronization
    pthread_cond_t cond;    // Thread sleeps here
```

```
    bool my_turn;           // Flag to wake up
    struct Job *next;       // Queue link
} Job;
```

2. Implementation Highlights

Core Flow

1. **Submission:** When a client sends a command, `client_thread_func` creates a Job, calculates the predicted burst (N for demo, -1 for shell), and calls `add_job()`.
2. **Wait:** The client thread locks the `sched_lock` and waits on `pthread_cond_wait(&j.cond)`.
3. **Selection:** The `scheduler_thread_func` wakes up, scans the linked list using `get_next_job()`, and signals the chosen job.
4. **Execution:** The client thread wakes up, executes for a specific **Quantum**, and then yields the CPU back to the scheduler.

Scheduling Algorithm (Combined SRJF + RR)

Our `get_next_job` function implements the logic:

1. **Shell Commands First:** It scans the list for any job with `is_shell_cmd == true`. These are run immediately (FCFS among themselves).
2. **SRJF for Programs:** If no shell commands exist, it finds the job with the minimum `remaining_time`.
3. **Anti-Monopoly Rule:** We implemented the check: `if (count > 1 && curr->id == last_job_id) skip`. This ensures that if multiple jobs are available, the same job is not selected twice in a row (unless it's the only one left).

Variable Quantum and Preemption

- **Variable Quantum:** Inside `client_thread_func`, we check `rounds_run`. If it is 0, we set `quantum = 3`. For all subsequent rounds, `quantum = 7`.

- **Preemption:** Inside `add_job`, if a new job arrives that has a shorter remaining time than the *currently running* job, we set `current_job->preempt_requested = 1`.

- **Interrupt Loop:** The execution loop reads output from the child process line-by-line. After every line (simulating 1 second), it checks `preempt_requested`. If true, it stops execution early, saving the state for the next round.

Timeline Generation

To verify correctness, we implemented a linked list `TimelineEntry`. Every time a job finishes a slice, `append_timeline()` records the job ID and duration. When the queue becomes empty, `print_timeline()` dumps the Gantt chart string (e.g., `0)-P6-(3)-P7-(6)...`) to the server console.

3. Execution Instructions

1. Compile: Run `make` in the project directory. This compiles server, client, and demo.

```
make
```

2. Start Server:

```
./server
```

3. Run clients: Open multiple *new* terminal windows. In each one, you can connect to the server.

```
# In Terminal 2
./client

# In Terminal 3
./client
```

4. Run Tests:

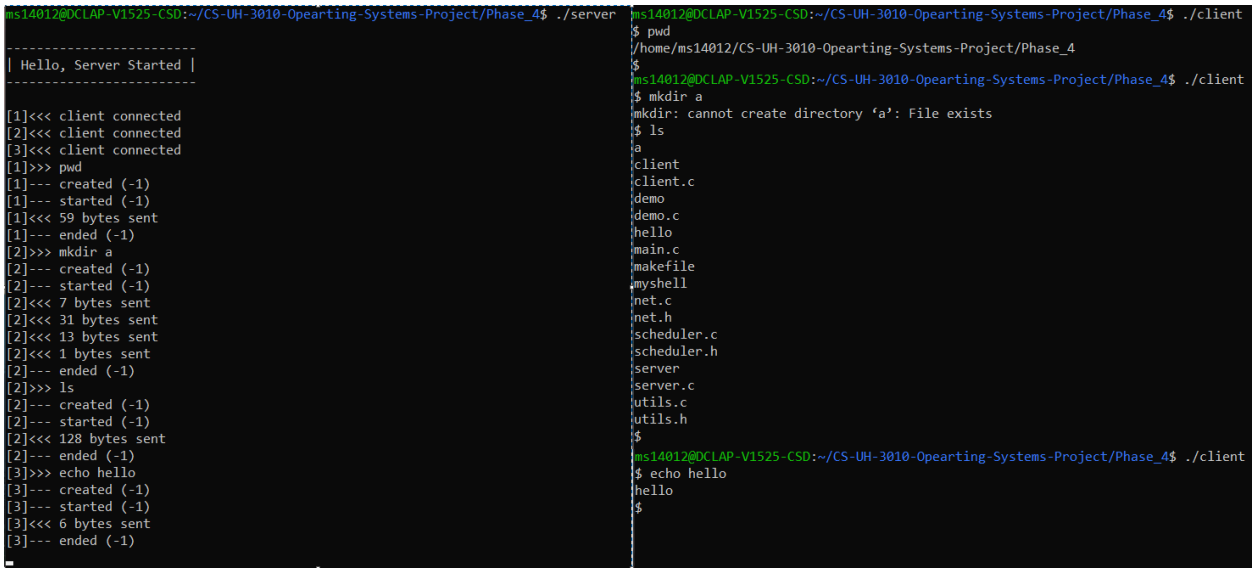
- Shell commands: `ls, pwd`

- Demo programs: `./demo 12`, `./demo 5`

4. Testing

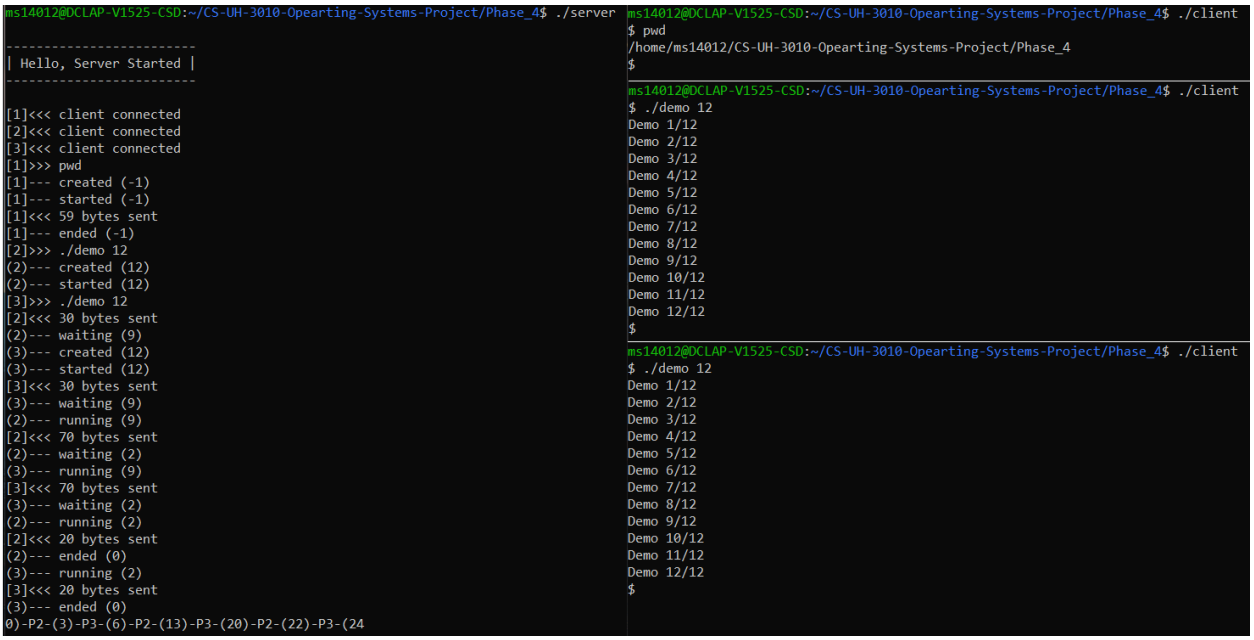
We validated the system using the three scenarios described in the project requirements. Please zoom in to the screenshots for more detail.

Test Scenario 1



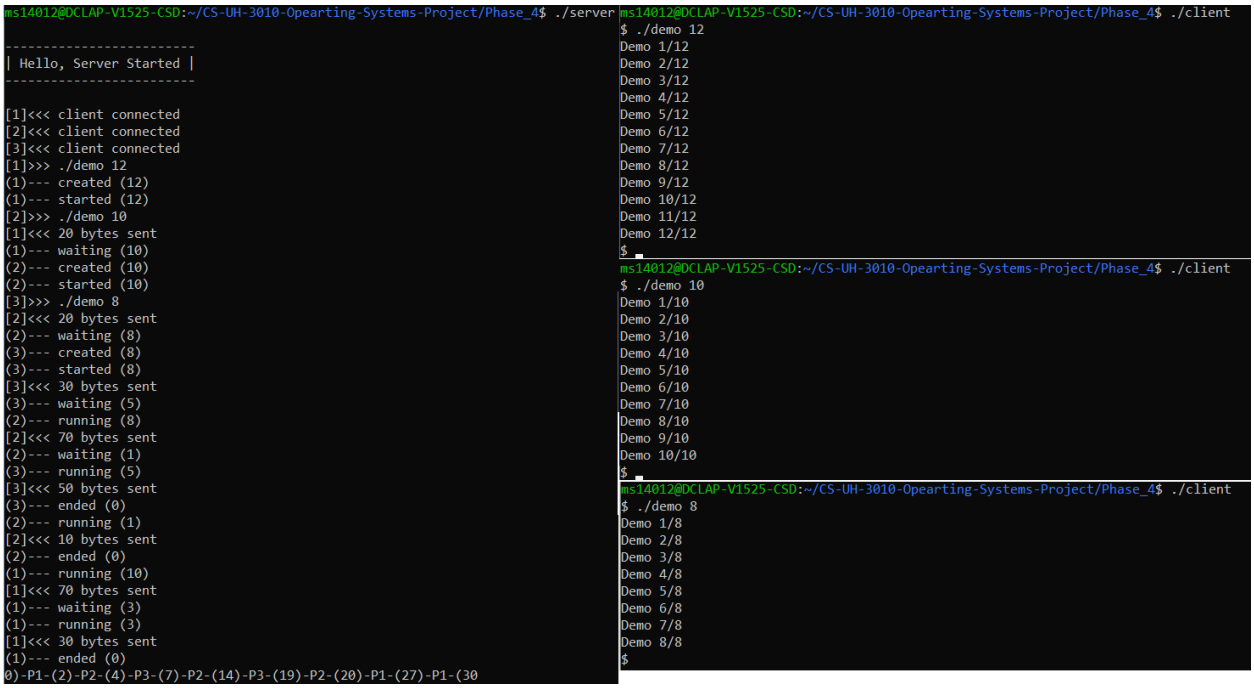
3 concurrent clients all running single commands

Test Scenario 2



3 concurrent clients: one client running a single command and the two others running demo

Test Scenario 3



3 concurrent clients: each client running the demo program with different values of N

5. Challenges

1. Handling Preemption Smoothly

One of the toughest parts was figuring out how to pause a running process without killing it. At first we tried just waiting, but that ended up blocking the whole scheduler.
Fix: We switched to using `pipe()` so we could read the child's output one line at a time and check for preemption between lines. We also used `SIGSTOP` to pause the child cleanly so it didn't lose any progress.

2. Enforcing a Single-CPU Abstraction

At one point, multiple jobs were effectively running at the same time, even though the project assumes a single CPU. When we unlocked the global scheduler mutex around execute_demo, the scheduler started handing the CPU to a second job while the first child process was still running.
Fix: We introduced a global cpu_busy flag and a current_job pointer, both protected by the scheduler mutex. The scheduler only picks a new job when !cpu_busy, and the client thread explicitly sets cpu_busy = false when its quantum ends or the job finishes. This kept the simulation logically single-core even though the implementation uses multiple threads.

3. Getting True SRJF Preemption (Not Just Time-Quantum Switching)

Initially, jobs only switched at the end of a time quantum. Even if a shorter job or a shell command arrived, the running program would finish its entire quantum before yielding, which violated the "shortest remaining job first" and "shell commands preempt everything" requirements.

Fix: We added a `preempt_requested` flag on the running job and made `execute_demo_job` cooperative: it reads one line at a time from the child's pipe and checks `preempt_requested` between lines. When a new job is enqueued, `add_job` compares remaining burst times (or detects a shell command) and sets `current_job->preempt_requested = 1` if preemption is needed. This allows immediate preemption at the next line boundary rather than only at quantum expiry.

6. Division of Tasks

Ayyub: Implemented the `scheduler.c` core logic, including the Job struct, the linked list queue, and the `get_next_job` algorithm (SRJF + RR). Handled the mutex/condition variable synchronization between the scheduler and client threads.

Matija: Implemented the process control logic (`fork`, `exec`, `SIGSTOP`, `SIGCONT`) and the variable quantum logic inside `server.c`. Wrote `demo.c` and the timeline generation/printing functions. Wrote the report.

7. References

Linux Programmer's Manual (man pages): `sigaction`, `kill`, `pthread_cond_wait`.